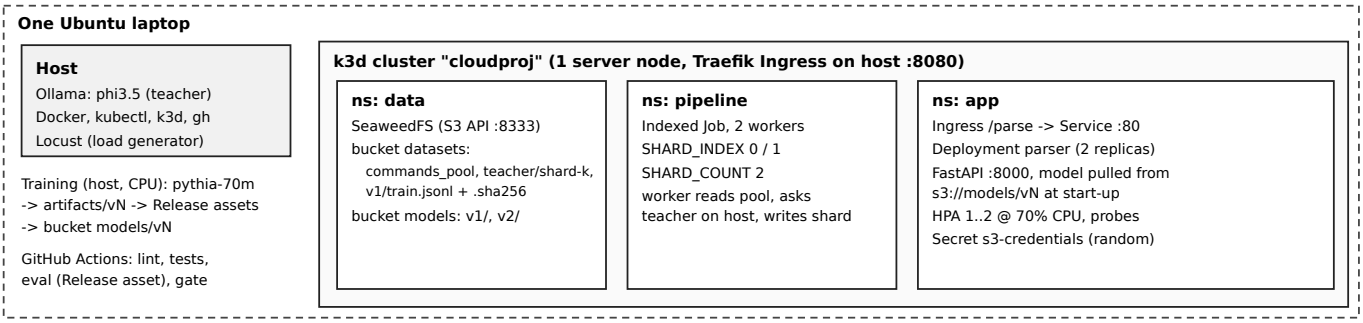
Knowledge Distillation and Deployment of a Command-Parsing Language Model on a Local Cloud

CS3081 Cloud Computing, Mini Project. Name: Himanshu Raj | Roll number: 230101084 | Repository: https://github.com/Himanshuraj884/cloud-project (tag `final`) | Release: https://github.com/Himanshuraj884/cloud-project/releases/tag/v1

[M] = **measured**: produced by a command whose output is stored in `results/` (file named). [A] = **assumption or interpretation**: not measured, or an explanation I did not test. All measurements were taken on one laptop (Ubuntu, 18 threads, CPU only) on 7 to 8 November 2026.

0. Architecture and decisions



Decisions. Sequence-level distillation, as required. `clean.py` only drops records and never repairs teacher text. The loss is taken on reply tokens only (the command part is masked). The service image uses the CPU-only PyTorch wheel and contains no weights; weights live only in object storage and Release assets. S3 keys are generated inside the cluster by `scripts/bootstrap_cluster.sh` and are never in Git. Everything is rebuilt by `make up`.

1. Task 1: Object storage and parallel teacher labelling

Implementation. `cluster/k3d.yaml` creates the cluster (1 server, host port 8080 to the load balancer). SeaweedFS runs in namespace `data` with buckets `datasets` and `models`; `commands_pool.jsonl` is uploaded to `datasets` (`results/models_bucket_listing.txt`). The teacher is Phi-3.5-mini (`phi3.5`) on Ollama on the host, temperature 0, seed 42, with my prompt in `prompts/teacher_prompt.txt` (four-key JSON, eight intents, three slot rules, six examples). Two workers run at the same time in namespace `pipeline` as an Indexed Job (`completions: 2, parallelism: 2`); each worker's `SHARD_INDEX` comes from the Job completion index and `SHARD_COUNT=2`.

Sharding (under half a page). The worker keeps command line number i (from 0) if i mod `SHARD_COUNT` = `SHARD_INDEX`. Worker 0 gets lines 0, 2, 4, ... and worker 1 gets 1, 3, 5, ...; every line has exactly one remainder, so there are no gaps and no repeats by construction. Each worker writes its own object (`datasets/teacher/shard-0.jsonl`, `shard-1.jsonl`), one line `{"id","text","teacher_raw"}` per command with the teacher reply stored exactly as received, and resumes from its own object after a restart. `scripts/verify_shards.py` checks that every id of the pool appears once across the shards (output in `results/verify_shards.txt`). [A] I did not measure how evenly the two workers shared the time; both read from the same single Ollama process.

A generation setting I added. With only temperature 0 and seed 42, Phi-3.5 often answered correctly and then kept inventing further "Command: ..." turns until the token limit, so replies were not a single JSON object. I added `num_predict=96` and stop sequences (`\n\n`, `Command:`) in `worker/teacher.py`, deleted the first labelling run and labelled again. Replies are still stored unedited. [M] The counts of the discarded first run were not kept.

Cleaning counts (`results/clean_counts.json`)	**Records**
Input records	3000
(a) reply not valid JSON	6
(b) keys not exactly intent, date, time, place	13
(c) intent not one of the eight	12
(d) slot neither null nor text appearing in the command	212
Duplicate commands (smallest id kept)	48
Kept (`datasets/v1/train.jsonl`)	**2709**

[M] Check: 6 + 13 + 12 + 212 + 48 + 2709 = 3000. Rules as implemented: checks run in the order a, b, c, d and each record is counted under the first reason it hits; duplicates are judged among the records that passed (a) to (d). Checksum in `train.sha256` = `f45855ba77c4cc72de89de3630d1ef067d01a91177cb84a07f73c634119533be`. Re-running `worker/clean.py` on the saved shard files (`results/teacher_shards/`) reproduced exactly this digest (`results/clean_local_check.txt`, "MATCH").

2. Task 2: Training the student (sequence-level distillation)

Implementation. `training/train.py` fine-tunes `EleutherAI/pythia-70m` (full fine-tuning, float32, CPU) on the first 1,500 records of `train.jsonl`. Example format: `Command: <command>\nJSON: <teacher reply>` + end-of-text. Everything it reads is in `training/config.yaml`: seed 42, learning rate 5e-4, batch size 16, 5 epochs, max sequence length 96, AdamW (weight decay 0), 5% linear warm-up then linear decay, gradient clipping 1.0, and library versions (Python 3.12, torch 2.4.1, transformers 4.44.2, numpy 1.26.4, PyYAML 6.0.2; the script stops if the installed versions differ). `make train` reads only that file; `make train SUBSET=50` trains on the first 50 records and ran to the end on my machine. The seed is set for Python, NumPy and torch and the data order uses a seeded generator.

Model	**Config**	**Loss per epoch (`training/loss*.csv`)**	**SHA-256 of release asset**
v1 (`models/v1/`)	seed 42, 5 epochs	`0.9137, 0.4435, 0.1956, 0.1058, 0.0459`	`c1afe5e8872f2b61818af091af06f2eb2a3ecdafe21ca04a77091f190996bd81`
v2 (`models/v2/`)	seed 44, 6 epochs	`0.9610, 0.4416, 0.2249, 0.1332, 0.0797, 0.0384`	`b737b47f490f64835841afd48f9ee8baa3e3808827371e97b375e241fb88330c`

[M] Both model + tokenizer archives, the checksum files and `version_info.json` are Release assets of tag `v1` and are also in bucket `models` under `v1/` and `v2/`. The checksum of the downloaded v1 asset equals `models/v1/model.sha256`; the same holds for v2 (checked with `gh release download`, since a plain `curl` briefly returned a cached older file).

Evaluation on `validation.jsonl` (100 commands, `tools/score.py`, percent) [M]

Model (raw output, report)	**json_validity**	**intent**	**date**	**time**	**place**	**full_frame**
Teacher Phi-3.5 (`teacher_predictions.jsonl`, `teacher_validation.json`)	98	81	70	84	87	42
Untrained Pythia-70M (`untrained_*`)	0	0	0	0	0	0
Student v1 (`student_*`), deployed by default	100	85	68	87	83	43
Student v2 (`v2_*`), used in the rolling update	100	88	74	89	88	54
v2 first attempt, seed 43 (`results/v2_attempt1/`), discarded	98	84	75	87	81	46

Discussion. [M] The untrained model never produces the required JSON (0% validity), so everything above 0 comes from distillation. The trained v1 matches the teacher closely (full frame 43 vs 42; intent +4, time +3, date -2, place -4 points) and is always valid JSON, while the teacher itself produced 2 invalid replies. [A] With only 100 validation examples one example is 1 point and the standard error of a 40 to 50% score is about 5 points, so the student should be read as roughly equal to the teacher, not better. A possible reason it does not fall below the teacher is that the 1,500 training targets passed the cleaning rules (slots must appear in the command), so the student learned from filtered, more

consistent replies than the raw teacher output; I did not test this. v2 scores higher (54), but it differs from v1 in two ways (seed and one more epoch), so I cannot say which caused the gain. I also chose seed 44 after seed 43 gave a broken reply on one command (V049: the date slot repeated "first of january one of" until the 64-token limit, giving invalid JSON, 98% validity). Picking a seed after seeing validation results is mild tuning on the validation set, so v2's score is optimistic; v1 was not re-selected.

Sequence-level versus logit-based distillation

(a) Sequence-level distillation trains the student with ordinary cross-entropy on the text the teacher generated (one hard target sequence per command), which is what I did. **(b)** Logit-based distillation trains the student to match the teacher's full next-token probability distribution at every position (soft targets, e.g. a KL-divergence loss), so it also learns which alternatives the teacher considered plausible. **(c)** It is not possible here because the teacher is served by Ollama, which returns only generated text (we store no logits), and Phi-3.5 and Pythia-70M use different tokenizers and vocabularies, so their distributions cannot be matched token by token without extra alignment. **(d)** **One real limitation of what I did:** the student can only learn the teacher's behaviour, including its mistakes, so the teacher's 42% full-frame accuracy is the practical ceiling for imitation; the cleaning step removes only mistakes that are detectable by rule (e.g. a slot not in the command), not wrong intents or wrong date choices. [M] Date accuracy is the weakest slot for v1 (68%).

3. Task 3: The inference service in a container

Implementation. `service/app.py` (FastAPI on uvicorn, port 8000). `POST /parse` takes `{"text": "..."}`, returns the four-key JSON with headers `X-Model-Version` (from `MODEL_VERSION`) and `X-Served-By` (Pod name from `HOSTNAME`); `GET /healthz` is 200 while the process is alive; `GET /readyz` is 503 until the model is loaded, then 200. Configuration is only `MODEL_URI`, `MODEL_VERSION`, `S3_ENDPOINT`, `MAX_NEW_TOKENS` (plus `FAULT_INJECT`); S3 credentials come from the Kubernetes Secret `s3-credentials` through the environment. At start-up a background thread downloads the model and tokenizer from object storage, then loads them. Every model output is checked by `service/validate.py` against `schema.json`; invalid output gives HTTP 500 with body `{"error": "invalid_model_output"}` and never HTTP 200. Generation is CPU-bound, so it runs in a worker thread under a lock (one generation at a time).

[M] **Fault injection** (`results/fault_inject_test.txt`): with `FAULT_INJECT=1` the service returned `HTTP/1.1 500 Internal Server Error` with body `{"error":"invalid_model_output"}`. **Image** (`results/image_size.txt`): base `python:3.11.9-slim`, CPU-only `torch==2.4.1` from the PyTorch CPU wheel index, all other dependencies pinned in `service/requirements.txt`, no weights; Docker reports 1.67 GB disk usage (359 MB content size), below the 2 GB limit. **50 sequential requests** (`scripts/mem_test.sh`, `results/memory_test.txt`) at `--cpus=1 --memory=1g`: all 50 returned HTTP 200, `OOMKilled: false`, memory about 396 to 399 MiB after each request, and the cgroup peak (`memory.peak`) was 744,263,680 bytes (about 710 MiB). [A] The peak is higher than the steady value, probably because it includes the model download and load at start-up and file cache; I did not break it down.

Challenge. My first HPA load test failed (kept in `results/run1_failed/`). Reading the service showed that generation ran on the async event loop, so `/healthz` and `/readyz` could not answer while a request was being generated; I moved generation to a thread (`run_in_threadpool`) and repeated the test (`results/hpa_run/`), which is the run reported below.

4. Task 4: Kubernetes deployment, scaling and reliability

Implementation (`k8s/app/deployment.yaml`, namespace `app`): Deployment `parser` with 2 replicas, requests 500m CPU / 768 Mi, limits 1 CPU / 1 Gi, liveness probe `/healthz`, readiness probe `/readyz`, rolling update `maxSurge 1 / maxUnavailable 0`, `preStop: sleep 10` and `terminationGracePeriodSeconds: 40`; Service port 80 to 8000; Traefik Ingress sending `/parse` to the Service; HPA min 1, max 2, target CPU 70%. [M] `/parse` answers through the Ingress on `localhost:8080` (`results/ingress_parse_test.txt`, and again from a fresh clone in `results/make_up_smoke.txt`).

Test	Measured result (evidence)	Requirement
No ready endpoint while the model loads	after a Pod was deleted, `ready_endpoint_ips=[]` from 22:18:42 to 22:18:47, the new Pod went `0/1 Running`, and the endpoint appeared at 22:18:48 (`endpoints_during_load.txt`; single-replica state, about 6 s)	Service has no ready endpoints while loading
HPA 1 to 2 to 1	CPU 0% on 1 replica; at 09:18:02 CPU 173% and `desired=2`; about 198 to 200% under load with 2 replicas; load ended about 09:19:50; `desired=1` at 09:24:38 and `current=1` at 09:24:58 (`hpa_watch.txt`, 2-minute standard test `hpa_run/`)	scale up under load, back down
Rolling update v1 to v2, then `rollout undo`, during an 8-minute load test	**0 failures of 2354 requests (0.00%)**; update started 16:13:31, undo 16:15:49; requests served: v1 1665, v2 689; afterwards 10 of 10 requests returned `X-Model-Version: v1` (`rollout_events.txt`, `rollout_stats.csv`, `after_rollback_check.txt`; per-replica counts in `rollout_headers_transcribed.txt`)	at most 0.5% failed; v1 after rollback
Recovery: delete one Pod under load (3 minutes)	Pod deleted at 16:34:09; ready replicas 2, then 1 (16:34:12 to 16:34:14), back to 2 at 16:34:16 (about 7 s); **0 failures of 854 requests** in the CSV (`recovery_events.txt`, `recovery_stats.csv`)	replicas restored; at most 2% failed
Cold start: delete all Pods, 3 runs	Pod creation to Ready: **8 s, 6 s, 8 s; median 8 s**; logs show `downloaded 7 model files from s3://models/v1/`, a 503 on `/readyz`, then `model v1 loaded, ready` (`coldstart.json`, `coldstart_logs.txt`)	three times and median, with log evidence
Load balancing, exactly 2 Ready replicas, standard test	344 and 344 successful requests (50% / 50%, 0 failures of 688) in `lb_run/`; the final Task 7 run gave 291 and 289 of 580	each replica at least 30%

What went wrong first. [M] My first rolling update (before the `preStop` hook, `results/rollout_run1_failed/`) had 31 failures of 1379 requests (2.25%): 6 HTTP 502, 24 HTTP 500, 1 connection error. [A] The 502s and the connection error are consistent with Traefik still sending requests to a Pod that was shutting down; the 24 HTTP 500s are consistent with the first v2 model (seed 43) giving invalid JSON for one validation command, but I did not map each failure to a request. Adding `preStop: sleep 10` and a longer grace period, plus the retrained v2, removed all failures in the run above. Single `curl` probes taken just after each rollout step in `rollout_events.txt` printed the older version (v1 after the update, v2 after the undo); [A] I believe a draining Pod still received that one request, so I added the 10-request check, which is the evidence for "v1 again". The per-replica counts of this run and of the recovery run were printed on screen only, because `locustfile.py` writes its header counts to one fixed file that later runs overwrote; I copied the printed numbers into the two `*_transcribed.txt` files and marked them as hand-copied.

Limitations (stated clearly). The cluster is one node on one laptop, so these tests show Pod-level recovery and balancing only, not fault tolerance across machines or zones. The HPA can only add replicas on the same laptop, its CPU percentage is relative to the 500m request (so about 200% means "at the 1-core limit"), and metrics-server readings on a laptop are coarse. [A] The slow scale-down (about 4 min 45 s after the load stopped) matches the Kubernetes default 5-minute stabilisation window; I did not change it. `kubectl rollout undo` warns that it does not update the `last-applied` annotation, so re-applying the manifest is the clean way to return to the file's state.

5. Task 5: Subnetting and network access control

Block 10.20.0.0/16, largest first, each subnet right after the previous one, smallest prefix that fits (usable hosts = $2^{32\text{-prefix}}$ - 2). [M] Verified with Python's `ipaddress` module (`results/subnets_check.txt`).

Subnet	Needed	Network	Prefix	Usable range	Broadcast	Usable hosts
app	1,000	10.20.0.0	/22	10.20.0.1 - 10.20.3.254	10.20.3.255	1022
public	250	10.20.4.0	/24	10.20.4.1 - 10.20.4.254	10.20.4.255	254
data	100	10.20.5.0	/25	10.20.5.1 - 10.20.5.126	10.20.5.127	126
management	20	10.20.5.128	/27	10.20.5.129 - 10.20.5.158	10.20.5.159	30

Why these prefixes: /23 would give only 510 hosts (too few for 1,000), /25 only 126 (too few for 250), /26 only 62 (too few for 100), /28 only 14 (too few for 20). **NACL for the public subnet** (`network/nacl.json`):

Dir	Rule	Action	Protocol / ports	Source or destination	Purpose
in	100	deny	all	198.51.100.0/24	abusive range, checked first
in	110	allow	TCP 80	0.0.0.0/0	HTTP
in	120	allow	TCP 443	0.0.0.0/0	HTTPS
in	130	allow	TCP 22	10.20.5.128/27 (management)	SSH only from management
in	140	allow	TCP 1024-65535	10.20.0.0/22 (app)	replies from the app subnet
out	100	allow	TCP 8000	10.20.0.0/22 (app)	load balancer to the service
out	110	allow	TCP 1024-65535	0.0.0.0/0	replies to clients

[M] `tools/nacl_sim.py network/nacl.json network/packets_sample.jsonl --check` decided all 6 sample packets correctly (6/6 = 100%, `results/nacl_check.txt`). [A] The marker uses 20 hidden packets; I assume the rules are right for those but only the six samples were measured.

Why a stateless NACL needs separate reply rules, and why order matters. A NACL does not remember connections: a reply is a new packet judged on its own against the rules for its direction. A client request in on port 443 is allowed by an inbound rule, but the response leaves from port 443 to the client's ephemeral port, so an outbound rule for TCP 1024-65535 to the clients is needed. Likewise the app subnet's replies to the load balancer arrive as inbound packets to ephemeral ports and need inbound rule 140. A stateful security group would allow these automatically. Rules are evaluated from the lowest number upward and the first match decides, with default deny. The deny for 198.51.100.0/24 is rule 100, lower than the allows for ports 80 and 443 (110, 120). If it were numbered above them, a packet from that range to port 443 would match the allow first and the block would never apply.

6. Task 6: Reproducibility, testing and CI/CD

Build and tear-down. `git clone <repo> && cd <repo> && make up` (three commands) creates the cluster, SeaweedFS and buckets, downloads both models from the Release, checks their SHA-256 against `models/*/model.sha256`, uploads them to `models/`, builds the image, and deploys Deployment, Service, Ingress and HPA. `make down` deletes the cluster. [M] Tested from a fresh clone of commit `bca457d`: `make up` finished in about 8 minutes (pod ages in `results/make_up_pods.txt`), the smoke request returned `{"intent": "alarm_set", "date": "tomorrow", "time": "seven am", "place": null}` (`make_up_smoke.txt`), and `make down` removed the cluster (`make_down_log.txt`, `make_down_check.txt` lists no cluster).

Two fresh-clone bugs found and fixed. (1) Right after the Ingress rollout the first request returned `no available server` (Traefik had not synced yet), so `scripts/up.sh` now waits until `/readyz` answers 200 through the Ingress. (2) One fresh run failed with "the server is currently unable to handle the request" because `kubectl apply` ran before the new cluster's API server was ready; `scripts/bootstrap_cluster.sh` now waits for the API, the node, metrics-server and Traefik. The earlier successful run was luck of timing.

CI (`.github/workflows/ci.yml`), stages in order: secret scan (gitleaks) in parallel with lint (`ruff`), then unit tests (12 tests in `tests/`: sharding, cleaning, validation, service endpoints including the exact fault-injection body), then evaluation (downloads the model from the GitHub Release, runs `eval/run_student.py` and `tools/score.py` on `validation.jsonl`), then the quality gate `score.py --min-validity 95`. [M] Passing run on the trained v1 (all five jobs succeeded, including gitleaks, which reported nothing): **https://github.com/Himanshuraj884/cloud-project/actions/runs/42475750397**. Deliberate failure with the untrained Pythia-70M from Release `baseline-untrained` (secrets, lint, unit-tests and evaluate succeeded, **quality-gate failed**, because the untrained model has 0% JSON validity): **https://github.com/Himanshuraj884/cloud-project/actions/runs/42475753028** (`results/ci_runs.txt`, `ci_failed_run_jobs.txt`). `AI_USE.md` lists the AI tools used, what for, and how I checked them.

7. Task 7: Performance, cost and review

Standard command (`make load-test`: 4 users, spawn rate 4, 2 minutes, host `http://localhost:8080`) against 2 Ready replicas through the Ingress; CPU and memory sampled every 10 s with `kubectl top pods` (`results/loadtest_*.csv`, `loadtest_headers.json`, `top_pods.txt`). [M]

Metric	Value
Requests / failures	578 / 0 (error rate 0.00%)
p50 and p95 latency	p50 740 ms (the median in the same file is 700 ms; Locust bins percentiles), p95 1300 ms (limit 5000 ms)
Throughput	4.854 requests per second
CPU per Pod during the test	parser-...-967j8: 940 to 964m; parser-...-ssmjt: 1000m (idle: 4m each); the limit is 1000m
Memory per Pod during the test	340 to 341 Mi and 349 to 350 Mi (limit 1 Gi)
Cold start	median 8 s (Task 4)
Requests per replica	291 and 289

Cost (`scripts/cost.py`, `results/cost.txt`; [A] the rates in `rate_card.json` are hypothetical, no real charge): per replica per hour = 1.0 vCPU x 3.5 + 1.0 GiB x 0.45 = 3.95 INR; 2 replicas = 7.90 INR per hour; requests per hour = 4.854 x 3600 = 17,474.7; **cost per 1,000 requests = 7.90 / 17,474.7 x 1000 = 0.4521 INR**.

SLO. Target: 99% of requests succeed, window: one load-test run (578 requests). Error budget = 578 x (1 - 0.99) = 5.8 requests. Failures observed = 0, so 0% of the budget was used. [A] One 2-minute run is a very short window; it shows the target was met in that run, not over time.

Component	Evidence (measured)	Limitation found	Improvement
Data and storage	2709 of 3000 records kept (90.3%); 212 dropped for slots not in the command	One SeaweedFS replica on one local volume; versions only by key prefix	Replicated volumes and bucket versioning; log rejected records for review
Model quality	v1 full frame 43% vs teacher 42%; date accuracy 68%; 100% valid JSON	Only 100 validation examples; student cannot exceed teacher's mistakes; seed for v2 chosen after seeing validation	Larger held-out set, several seeds with mean and spread; train on the whole cleaned set within the limit; add a logit-style or repair step
Latency and throughput	p95 1300 ms, 4.85 req/s with 2 replicas, 0 errors	One generation at a time per Pod; CPU saturated, so throughput is CPU-bound	Shorter `MAX_NEW_TOKENS`, batching, quantised or ONNX model, more replicas on more nodes
Resource utilisation	CPU 940 to 1000m of 1000m; memory 341 to 350 Mi of 1 Gi; HPA read about 200% (relative to the 500m request)	CPU limit is the bottleneck; memory limit is far above use; HPA percent depends on the request	Raise CPU limit or lower per-request work; right-size memory to about 768 Mi; set the request from measured use
Reliability and recovery	Rolling update 0 of 2354 failed; Pod replaced in about 7 s; cold start median 8 s; 0 of 854 failed during recovery	Single node; no protection against node or laptop loss; first rollout without `preStop` failed 2.25%	Multi-node cluster, PodDisruptionBudget, topology spread, longer test windows
Security and cost	No credentials in Git (gitleaks job green); random S3 keys live only in cluster Secrets; non-root container user; 0.4521 INR per 1,000 requests (hypothetical rates)	S3 traffic is plain HTTP inside the cluster; Secrets are base64, not encrypted; one shared admin key; cost uses assumed rates	TLS and per-service least-privilege keys, encrypted Secrets, NetworkPolicies, cost from real billing data

8. Assumptions and honest limits (summary)

- [A] Rate card values are hypothetical; the cost figure is an estimate only.
- [A] The 100-example validation set is only a proxy for the hidden 300-example test set; differences of a few points are within noise.
- [A] Explanations marked above (why the student matches the teacher, why single probes showed the old version, why scale-down took about 5 minutes, what the cgroup peak includes) were not tested.
- [M] The teacher setting includes stop sequences and a 96-token limit beyond temperature 0 and seed 42; the first labelling run without them was discarded and its counts were not kept.

- [M] Per-replica counts for the rollout and recovery runs were copied by hand from the console into `results/*_transcribed.txt`; the Locust CSV files for these runs are the original outputs. Failed or discarded attempts are kept in `results/run1_failed/`, `rollout_run1_failed/`, `attempt1_no_deletion/` and `v2_attempt1/`.
- Single-node k3d on one laptop: no multi-machine fault tolerance and no real cloud cost.

References

- Kim and Rush, "Sequence-Level Knowledge Distillation", EMNLP 2016. https://aclanthology.org/D16-1139/
- Hugging Face Transformers, causal language modeling guide; EleutherAI Pythia-70M model card. https://huggingface.co/EleutherAI/pythia-70m
- k3d, k3s (Traefik), SeaweedFS, Kubernetes documentation (Deployments, probes, HPA, rolling update); Locust; gitleaks; AWS VPC Network ACLs user guide; Google SRE Book, Service Level Objectives.

Appendix A: Evidence index (all in the repository, not counted in the page limit)

Task	Files in results/ (and other paths)
1	clean_counts.json, clean_output.txt, clean_local_check.txt, verify_shards.txt, teacher_shards/, models_bucket_listing.txt; prompts/teacher_prompt.txt; data/train.sha256
2	training/loss.csv, loss_v2.csv, config.yaml, config_v2.yaml; models/v1|v2/model.sha256, version_info.json; teacher|untrained|student|v2_predictions.jsonl and *_validation.json; v2_attempt1/
3	fault_inject_test.txt, memory_test.txt, image_size.txt
4	ingress_parse_test.txt, endpoints_during_load.txt, hpa_watch.txt, hpa_run/, lb_run/, rollout_stats.csv, rollout_events.txt, rollout_headers_transcribed.txt, after_rollback_check.txt, recovery_stats.csv, recovery_events.txt, recovery_headers_transcribed.txt, coldstart.json, coldstart_logs.txt, k8s_state.txt; failed attempts in run1_failed/, rollout_run1_failed/, attempt1_no_deletion/
5	nacl_check.txt, subnets_check.txt; network/subnets.md, nacl.json
6	make_up_log.txt, make_up_smoke.txt, make_up_pods.txt, make_down_log.txt, make_down_check.txt, ci_runs.txt, ci_failed_run_jobs.txt; .github/workflows/ci.yml, AI_USE.md
7	loadtest_stats.csv, loadtest_headers.json, loadtest_stats_history.csv, top_pods.txt, cost.txt

Appendix B: Commands

Build/deploy: **make up** Tear down: **make down** Test: **make lint && make test** Load test: **make load-test**
Data: **make label**, **make clean-data**, **make fetch-train** Train: **make train [SUBSET=50]**, **make train-v2**, **make package V=v1|v2** Evaluate: **make eval-teacher**, **make eval**
Rolling update: **kubectl -n app set env deploy/parser MODEL_URI=s3://models/v2/ MODEL_VERSION=v2**, then **kubectl -n app rollout undo deploy/parser**